

Αναφορά Εργασίας

Main :

Το πρόγραμμα ξεκινάει ελέγχοντας τον αριθμό των arguments που έδωσε ο χρήστης για να βεβαιωθεί ότι είναι 2 ο αριθμός των πελατών και ένας τυχαίος σπόρος πριν ξεκινήσει. Στη συνέχεια βάζει το πρώτο argument (αριθμός πελατών) σε μια μεταβλητή με όνομα `maxnumofthreads` και το δεύτερο (τυχαίος σπόρος) σε μια μεταβλητή με το όνομα `seed`. Συνεχίζει κάνοντας τους απαραίτητους ελέγχους για να βεβαιωθεί ότι και οι 2 μεταβλητές είναι θετικές βγάζοντας μήνυμα λάθους και τελειώνοντας το πρόγραμμα σε κάθε άλλη περίπτωση. Μετά από τους ελέγχους ο αριθμός των πελατών μπαίνει στη μεταβλητή `Ncust` και δεσμεύουμε μνήμη για να δημιουργήσουμε τον κατάλληλο αριθμό από `pthread_t` κάθε φορά ανάλογα με τον αριθμό πελατών που μας δόθηκε. Ελέγχουμε αν υπάρχει αρκετή μνήμη και αν δεν υπάρχει τερματίζουμε το πρόγραμμα.

Έχουμε δημιουργήσει ένα struct `cust_info` στο οποίο υπάρχουν 2 στοιχεία , το `id` και το `radomn` για να δημιουργήσουμε ένα πίνακα `Customers_Info` μεγέθους `Ncust` που κάθε στοιχείο του αντιστοιχεί σε 1 πελάτη και κουβαλάει το `id` του και έναν τυχαίο αριθμό ο οποίος θα χρησιμοποιηθεί αργότερα στη γεννήτρια τυχαίων αριθμών. Αρχικοποιούμε τα `mutexes` και `conditions` που θα χρειαστούμε για την διαχείριση των threads κάνοντας πάντα τους απαραίτητους ελέγχους για τυχόν σφάλματα.

Στη συνέχεια αρχίζει η δημιουργία των threads περνώντας τα απαραίτητα γνωρίσματα (σαν `id` έχουμε το `i+1` και σα τυχαίο αριθμο το `seed+i+1` για κάθε πελάτη) και περιμένουμε ένα μικρό τυχαίο χρονικό διάστημα ανάμεσα στην εμφάνιση του κάθε πελάτη. Γίνεται ο έλεγχος για πιθανά errors και έπειτα συνεχίζουμε κάνοντας `join` τα threads.

Τελειώνοντας εμφανίζουμε τα τελικά στατιστικά , καταστρέφουμε τα `conditions` και `mutexes` και ελευθερώνουμε τη μνήμη που έχουμε δεσμεύσει κάνοντας ελέγχους για errors σε κάθε μία από τις παραπάνω διαδικασίες.

Order :

Αρχίζουμε με την αρχικοποίηση ορισμένων μεταβλητών όπως το `time_ready` που θα δείχνει σε ποιο χρονικό σημείο βρισκόμαστε κάθε φορά στην παραγγελία την `info` που κρατάει τα στοιχεία του πελάτη που περάσαμε μέσα στη μέθοδο και την `cold` που θα μετράει το χρόνο από το ψήσιμο του φαγητού μέχρι την παράδοσή του. Για να μην επαναλαμβανόμαστε κάθε φορά που ανοίγουμε ή κλείνουμε ένα `mutex` ή χρησιμοποιούμε `condition` γίνονται οι απαραίτητοι έλεγχοι για errors. Επίσης κάθε

φορά που εμφανίζουμε κάποιο μήνυμα πριν την εμφάνισή του κλειδώνουμε το OutputMutex και αφού εμφανιστεί το ξεκλειδώνουμε.

Πρώτα από όλα κλειδώνουμε το OrderMutex πριν κάνουμε τον έλεγχο για το αν υπάρχουν διαθέσιμοι τηλεφωνητές. Ο έλεγχος γίνεται με τη χρήση επανάληψης όπου ελέγχουμε αν οι τηλεφωνητές είναι 0 και αν αυτό είναι αλήθεια εκτυπώνουμε «The order with number (id) is waiting in line» και στη συνέχεια χρησιμοποιούμε το condition OrderCond για να περιμένει το πρόγραμμα μέχρι να ειδοποιηθεί ότι κάποιος τηλεφωνητής έγινε διαθέσιμος. Αφού γίνει αυτό μειώνουμε τους τηλεφωνητές αφού αυτός που έγινε διαθέσιμος τώρα βρίσκεται στην παραγγελία που είμαστε. Εκτυπώνουμε το μήνυμα «The order with number (id) is on call» και κάνουμε generate ένα τυχαίο αριθμό από πίτσες (pizzas) ανάμεσα στα νούμερα Norderhigh Norderlow. Συνεχίζουμε δημιουργώντας 3 μεταβλητές που η κάθε μια αντιπροσωπεύει τον αριθμό από πίτσες που πήρε ο πελάτης για κάθε είδος. Τον αριθμό για κάθε είδος τον βρίσκουμε μέσω μίας επανάληψης όπου για κάθε πίτσα φτιάχνουμε ένα τυχαίο αριθμό και αν αυτός ο αριθμός %100 είναι μικρότερος από την πιθανότητα πίτσας μαργαρίτα επί 100 τότε αυξάνουμε τις πίτσες σπέσιαλ αλλιώς αν είναι μικρότερος της πιθανότητας πεπερόνι επί 100 αυξάνουμε τον αριθμό πεπερόνι αλλιώς αυξάνουμε τον αριθμό σπέσιαλ. Αθροίζουμε τις πίτσες σε μια μεταβλητή total και ξεκλειδώνουμε το OrderMutex για να κοιμηθεί το πρόγραμμα ένα τυχαίο χρόνο ανάμεσα σε Traymenthigh Traymentlow όσο πληρώνει ο πελάτης. Έπειτα κλειδώνουμε ξανά το mutex και υπολογίζουμε την πιθανότητα η παραγγελία να απέτυχε με ανάλογο τρόπο όπως υπολογίσαμε και τις από πάνω πιθανότητες. Αν έχει αποτύχει εμφανίζουμε «The order with number (id) failed» αυξάνουμε τον αριθμό των τηλεφωνητών, των Failed_Orders, κάνουμε signal το OrderCond για να ενημερώσουμε ότι υπάρχει διαθέσιμος τηλεφωνητής, ξεκλειδώνουμε το mutex, σταματάμε να μετράμε το χρόνο και κάνουμε pthread_exit. Αντίθετα αν η πληρωμή πετύχει εμφανίζουμε «The order with number (id) passed», αυξάνουμε το Passed_Orders, υπολογίζουμε την τιμή από τις πίτσες και την προσθέτουμε στο Total_Selling_Profit, αυξάνουμε τον συνολικό αριθμό κάθε είδους πίτσας με τους αριθμούς της συγκεκριμένης παραγγελίας, κάνουμε τον τηλεφωνητή διαθέσιμο και ειδοποιούμε το condition με τη χρήση του signal και παίρνουμε το χρόνο που είχαμε αρχίσει στον έλεγχο για τηλεφωνητή και τον προσθέτουμε στον συνολικό time_ready.

Με παρόμοιο τρόπο γίνεται και η διαδικασία της προετοιμασίας. Κλείνουμε το CookMutex ξεκινάμε το χρόνο, κάνουμε τον έλεγχο για μάγειρες βγάζοντας μηνύματα ότι η παραγγελία περιμένει για μάγειρα και το πρόγραμμα περιμένει με τη χρήση του wait και του CookCond. Όταν βρεθεί μειώνουμε το Ncook και εμφανίζουμε ότι η προετοιμασία έχει αρχίσει. Σταματάμε το μέτρημα του χρόνου, προσθέτουμε το χρόνο που πήρε η αναμονή για μάγειρα και η προετοιμασία στο time_ready, ξεκλειδώνουμε το mutex και αφήνουμε να κοιμηθεί το χρόνο της προετοιμασίας δηλαδή total*Trprep.

Ξεκινάμε με τους φούρνους αφού οι πίτσες έχουν ετοιμαστεί. Κλειδώνουμε το OvenMutex, ξεκινάμε το χρόνο, περιμένουμε για φούρνους να γίνουν διαθέσιμοι με

τη χρήση του OvenCond αν δεν είναι ήδη βγάζοντας τα κατάλληλα μηνύματα. Όταν γίνουν διαθέσιμοι μειώνουμε όσους πιάστηκαν και εμφανίζουμε ότι η παραγγελία ψήνεται. Σταματάμε το χρόνο και τον προσθέτουμε στο time_ready. Ξεκλειδώνουμε το mutex , κοιμάται το πρόγραμμα για χρόνο ψησίματος , κλειδώνουμε και το oven αλλά και το cook mutex γιατί δεν έχουμε ελευθερώσει ακόμα το μάγειρα , προσθέτουμε το Tbakе στο time_ready αυξάνουμε τους μάγειρες, κάνουμε signal με το CookCond και ξεκλειδώνουμε το CookMutex και OvenMutex.

Τελειώνοντας μένει το πακετάρισμα και το ντελίβερι. Κλειδώνουμε το DeliveryMutex , περιμένουμε για διαθέσιμους deliverers με τη χρήση του wait και DeliveryCond ,εμφανίζουμε κατάλληλα μηνύματα και όταν κάποιος γίνει διαθέσιμος τον αφαιρούμε για να παραδώσει την παραγγελία που βρισκόμαστε. Σταματάμε το χρόνο και τον προσθέτουμε στο time_ready όπως επίσης και το χρόνο για το πακετάρισμα total*Track. Αυτό το χρόνο τον προσθέτουμε και στον cold (χρόνος που κρυώνουν). Ξεκλειδώνουμε το mutex , sleep(total*Track) , το ξανακλείνουμε και εμφανίζουμε το χρόνο στον οποίο η παραγγελία ήταν έτοιμη . Αυτή τη φορά κλείνουμε και το OvenMutex για να ελευθερώσουμε τους φούρνους και να κάνουμε signal το OvenCond. Ξεκλειδώνουμε το OvenMutex , κάνουμε generate ένα τυχαίο αριθμό ανάμεσα σε Tdelhigh Tdellow , ξεκλειδώνουμε και κλειδώνουμε το DeliveryMutex για να κοιμηθεί για χρόνο transport , τον προσθέτουμε στο time_ready και στο cold. Αυξάνουμε το avg_srv για time_ready ώστε να προσθέσουμε στο συνολικό χρόνο το χρόνο που η παραγγελία ήταν έτοιμη και είχε μεταφερθεί. Ελέγχουμε αν το time_ready ήταν μεγαλύτερο από τον μαx χρόνο εξυπηρέτησης και αν ναι το αλλάζουμε. Το ίδιο κάνουμε και με τον max_cold με τον cold. Στο avg_cold προσθέτουμε τον cold, εμφανίζουμε το χρόνο παράδοσης, ξεκλειδώνουμε το mutex , κοιμάται για το χρόνο επιστροφής του deliverer, κλειδώνουμε ξανά, αυξάνουμε το Ndeliverer κάνουμε signal το DeliveryCond και ξεκλειδώνουμε το mutex.